

Documentation Technique - Stubborn E-Commerce

DOCUMENTATION TECHNIQUE - STUBBORN E-COMMERCE

Introduction

Cette documentation décrit l'architecture technique, les choix d'implémentation et les procédures d'installation et de test de l'application e-commerce **Stubborn**. Développée sous le framework PHP **Symfony 8.1**, cette application permet à des clients d'acheter des sweat-shirts premium en ligne et à un administrateur de gérer le catalogue et les stocks physiques.

1. Architecture de l'Application

1.1 Modèle de Données (Doctrine ORM)

La base de données repose sur trois tables principales modélisées de manière optimale pour répondre aux exigences du cahier des charges :

- **User** : Représente les utilisateurs de l'application. Elle implémente `ManagerInterface` et `PasswordAuthenticatedManagerInterface` de Symfony.
 - *Attributs majeurs* : `id`, `email`, `password` (haché), `name` (nom complet requis pour la livraison), `deliveryAddress` (adresse de livraison), `roles` (ex : `ROLE_USER` ou `ROLE_ADMIN`), et `isVerified` (booléen indiquant si l'adresse email a été confirmée).
- **Product** : Représente les sweat-shirts mis en vente.
 - *Attributs majeurs* : `id`, `name`, `price` (décimal), `image` (nom du fichier photo sur le serveur), et `isFeatured` (booléen pour la mise en avant en page d'accueil).
- **ProductStock** : Gère la quantité de stock disponible de manière granulaire.
 - *Relation* : Liaison bidirectionnelle **Many-To-One** avec l'entité `Product` (un produit possède plusieurs lignes de stock correspondant aux différentes tailles, mais une ligne de stock n'appartient qu'à un seul produit).
 - *Attributs majeurs* : `id`, `size` (tailles strictes : XS, S, M, L, XL), `quantity` (quantité entière en stock) et `product`.

1.2 Services Métiers Dédiés

Pour assurer une séparation stricte des responsabilités (principe SOLID), la logique métier est déportée dans des services injectables :

A. CartService (`src/Service/CartService.php`)

Ce service gère la session utilisateur pour y stocker les articles ajoutés.

- **Structure de stockage** : Les articles sont stockés sous forme de clé unique combinée ID_PRODUIT-TAILLE associée à la quantité sélectionnée. Exemple :

```
[  
    "12-M" => 2, // 2 exemplaires du produit ID 12 en taille M  
    "15-XL" => 1 // 1 exemplaire du produit ID 15 en taille XL  
]
```

- **Fonctionnalités** :
 - add(id, size) : Ajoute ou incrémente la quantité pour un produit et une taille donnés.
 - remove(id, size) : Supprime définitivement la ligne du panier.
 - getDetailedCart() : Récupère le panier brut en session, interroge le dépôt Doctrine pour charger les objets Product réels et calcule les sous-totaux par ligne.
 - getTotal() : Calcule la somme financière cumulée de tout le panier.
 - clear() : Vide le panier en session.

B. StripeService (src/Service/StripeService.php)

Ce service encapsule toute l'interaction avec le SDK PHP officiel de Stripe.

- **Fonctionnalités** :
 - createCheckoutSession(cartItems, successUrl, cancelUrl) : Traduit le panier détaillé dans la structure exigée par Stripe, convertit les prix décimaux en centimes d'euros (entiers requis par Stripe) et génère l'URL sécurisée vers la page Stripe Checkout de paiement.
 - retrieveSession(sessionId) : Interroge les serveurs Stripe pour récupérer le statut final de la transaction lors du retour utilisateur.

2. Choix Techniques & Justifications

- **Symfony 8.1 & PHP 8.5** : L'utilisation de versions modernes permet de bénéficier des arguments nommés dans les annotations de formulaires et de validation (ex : new Length(min: 6, ...)) éliminant l'ancienne syntaxe par tableaux d'options dépréciée.
- **Paieement sécurisé en mode TEST (Sandbox)** : L'application utilise la passerelle de paiement **Stripe Checkout** configurée exclusivement en mode développement/test. Aucune transaction financière réelle n'est effectuée. Les données bancaires des clients ne transitent jamais sur nos serveurs et ne sont pas stockées en base de données (conformité aux standards PCI-DSS).
- **Sessions PHP natives pour le panier** : Le panier est stocké en session utilisateur côté serveur. Ce choix évite des requêtes en base de données répétées avant que le client n'ait validé son achat et assure une isolation naturelle des paniers par navigateur.
- **Décrémentation défensive des stocks** : La décrémentation des stocks physiques dans ProductStock est effectuée de manière sécurisée lors du retour sur la page /cart/success uniquement si Stripe confirme que la transaction a le statut paid. Un garde-fou empêche également que le stock ne devienne négatif.
- **SQLite pour l'environnement de test** : Pour garantir des tests rapides et isolés, Doctrine est configuré en environnement de test pour utiliser une base de données locale SQLite par fichier

(test.db localisé dans le dossier de cache temporaire).

3. Instructions d'Installation et de Lancement

Suivez les étapes ci-dessous pour déployer et exécuter l'application localement sur votre machine.

3.1 Prérequis

- PHP 8.2 ou supérieur (PHP 8.5 installé sur la machine de test)
- Composer
- Un serveur de base de données MySQL
- Symfony CLI (recommandé)

3.2 Déploiement

1. **Cloner le dépôt** du projet et se placer à la racine du dossier.
2. **Installer les dépendances** PHP via Composer :

```
composer install
```

3. **Configurer le fichier d'environnement local** : créer un fichier `.env.local` à la racine du projet et y renseigner vos configurations de base de données MySQL et vos clés Stripe Test :

```
# URL de connexion à votre MySQL
DATABASE_URL="mysql://root@127.0.0.1:3306/stubborn?
serverVersion=8.0&charset=utf8mb4"
```

```
# Transport de courriers
MAILER_DSN=null://null
```

```
# Clés Stripe de test
STRIPE_PUBLIC_KEY=pk_test_votre_cle_publique_stripe
STRIPE_SECRET_KEY=sk_test_votre_cle_secrete_stripe
```

4. **Initialiser la base de données** : exécuter les commandes suivantes dans l'ordre pour créer la structure et charger les sweat-shirts et utilisateurs par défaut :

```
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --no-interaction
```

5. **Lancer le serveur de développement** : démarrer le serveur Symfony CLI local :

```
symfony serve
```

L'application est désormais accessible à l'adresse :
http://127.0.0.1:8000

4. Suite de Tests et Couverture

L'application intègre des tests unitaires et fonctionnels pour sécuriser le code du panier et le tunnel de paiement.

4.1 Lancement des tests en console

Pour exécuter la suite de tests automatisés, lancez la commande suivante dans votre terminal :

```
vendor/bin/phpunit
```

4.2 Résultats attendus en console

Tous les tests doivent passer au vert. La console renvoie le rapport suivant :

```
PHPUnit 13.2.4 by Sebastian Bergmann and contributors.
```

```
Runtime:      PHP 8.5.8  
Configuration: phpunit.dist.xml
```

```
.....  
7 / 7 (100%)
```

```
Time: 00:00.257, Memory: 42.50 MB
```

```
OK (7 tests, 29 assertions)
```

Chaque point (.) correspond à un scénario de test réussi sans erreur ni avertissement.

4.3 Détail des scénarios testés

CartServiceTest (tests unitaires du panier) :

1. Panier initial vide.
2. Ajout simple d'un produit.
3. Incrémentation automatique de la quantité (même produit, même taille).
4. Distinction par taille (même produit, tailles différentes).
5. Retrait d'un article spécifique.
6. Calcul exact du montant total.

CartControllerTest (test fonctionnel de l'achat) :

7. Simulation d'un parcours d'achat complet avec un mock du service Stripe : vérification de la décrémentation du stock en base de données et du vidage du panier après confirmation du paiement.